C-Agent: A Terminal AI Coding Agent

CS2313 — Operating Systems Course Project

Hengtao Wu 524031910038 GitHub

June 2026

1 Architecture Overview

C-Agent is a pure-C terminal coding assistant that follows the **ReAct** (Reason + Act) loop pattern. The user types a natural-language request; the agent forwards it to a remote LLM, executes whatever tools the LLM requests, feeds the results back, and prints the final answer. Although the handout only requires a basic coding agent, I intentionally treated the project as a small operating-system runtime for AI agents: it schedules tool calls, isolates file access, persists sessions, reclaims context memory, and exposes an extensible interface for future capabilities.

1.1 Module Map

Module	Key Files	Responsibility
Agent	`agent/agent.c`	ReAct loop, context reclaim, UI sync
LLM Client	`agent/llm_client.c`	HTTP request construction, JSON response parsing
Context	`context/`	Token budget tracking, offload & summary policies
Executor	`tools/executor.c`	Serial / parallel tool dispatch
Registry	`tools/registry.c`	ToolDef dispatch table
Sandbox	`tools/sandbox.c`	Path containment via `realpath`
Session	`session.c`	Checkpoint + append-only log persistence
Memory	`memory.c`	6-type typed memory, proactive remember, manual consolidation
Skills	`skills.c`	Progressive skill discovery and loading
UI	`ui/`	Render thread, spinner, markdown output
Hooks	`hooks.c`	Tool lifecycle event bus

1.2 Data Flow

A single user request traverses the system as follows:

1. `main.c` reads input; `cmd_dispatch` handles `/` commands.
2. `agent_chat` pushes the user message into the `Context`.
3. `ctx_reclaim` checks token budget; triggers offload/summary if needed.
4. `llm_chat` builds an HTTP/1.1 POST to `/api/v1/chat/completions`, including all registered `ToolDef` schemas discovered from the registry.
5. While the LLM returns tool calls: `executor_run_tools` dispatches them (parallel if all read-only), pushes results, reclaims context, and calls the LLM again.
6. The final text reply passes through `ui_idle()` (barrier) then `ui_print_markdown()`.

2 Key Design Decisions

2.1 Dispatch Table for Tool Registry

Inspired by OS syscall tables and VFS operation tables, each tool is a self-contained `ToolDef` struct bundling name, description, JSON schema, execution function pointer, and a `read_only` scheduling hint. The registry is a flat array populated once at boot; at runtime, tool lookup is a linear scan

over at most 16 entries. This design means *adding a new tool requires zero changes to the agent loop or LLM client*—a critical property as the project grew from 1 tool (Part 1) to 16 tools (Part 3 + extensions).

ToolDef interface (tools/tools.h)

```c
typedef ToolResult (*ToolFn)(cJSON *args);

typedef struct {
    const char *name;
    const char *desc;
    const char *param_schema;  // JSON Schema string
    ToolFn exec;
    bool read_only;            // scheduling metadata
} ToolDef;
```

2.2 Conservative Parallel Scheduling

The executor uses a simple but correct rule: if *all* tools in a batch are registered and `read_only`, dispatch them as concurrent pthreads; otherwise fall back to serial execution. Each task writes only to its own slot `tasks[i].result`; the main thread reads all slots only after every worker joins. This preserves the **request-order invariant** (`out_msgs[i]` corresponds to `tool_calls[i]`) regardless of completion order—analogous to how CPUs may execute out-of-order but retire in-order.

2.3 Two-Layer Context Reclamation

Context management uses a policy engine with pluggable strategies:

- **Offload policy** (lossless): When token usage exceeds `OFFLOAD_THRESHOLD` (default 0.8), large tool outputs are written to session-local files under `.agent/offload/` and replaced with compact placeholders containing recovery hints. The policy also maintains an offload manifest recording each artifact's path, size, and source message, so the agent can inspect what was paged out and call `read_file` to retrieve the full content on demand.
- **Summary policy** (lossy): When usage exceeds `SUMMARY_THRESHOLD`, a dedicated LLM call compresses old messages into a structured handoff with sections like `Goal`, `Completed`, `Next Steps`.

Offload runs first because it is lossless and cheap; summary handles the remaining verbosity. This staged design mirrors virtual memory's *page-out before OOM-kill* philosophy.

2.4 Sandbox: `realpath` over String Checks

Path validation uses `realpath(3)` rather than string-level `".."` detection. This correctly handles symlink attacks, paths like `foo/../../etc/passwd`, and absolute paths outside the workspace. For write targets that do not yet exist, the parent directory is resolved and checked, then the leaf is grafted back by string concatenation.

2.5 Session Persistence: Checkpoint + Append-Only Log

Sessions are organized as directories under `.agent/sessions/<session_id>/`. Each directory contains metadata for UI display, a `checkpoint.json` full snapshot, and an append-only `log.jsonl` delta stream. Recovery replays the log on top of the latest checkpoint, while automatic checkpointing every 10 messages bounds recovery time. This directory-level organization also gives context offload a session-local namespace, so offloaded artifacts from different conversations do not get mixed together. All writes are atomic (write to `.tmp`, then `rename`) to survive crashes mid-write.

3 Extensions Beyond the Handout

The extra features were not added simply to increase the tool count. They were chosen to make the agent usable as a long-running development assistant. In particular, session persistence makes the REPL recoverable, memory preserves stable facts across conversations, skills reduce system-prompt bloat, web tools let the agent inspect current documentation, hooks decouple cross-cutting behavior from the executor, and markdown rendering makes the terminal output readable enough for daily use. The system prompt was also extended into a dynamic construction pipeline that injects selected memory, skill manifests, context-recovery hints, and web-use guidance; the detailed prompt design is shown in the appendix PDF.

At a higher level, these extensions turn the project from a one-shot ReAct demo into a **stateful agent runtime**. The design adds five production properties that are not usually visible in minimal agents: durable identity (memory), durable execution history (sessions), bounded context growth (offload/summary), capability modularity (skills and tool registry), and operational observability (hooks and synchronized UI). This framing was useful because each feature supports a system-level invariant rather than existing as an isolated bonus command.

Extension	Design Summary
SubAgent	Thread pool (max 8). Each child is a `silent` Agent with its own Context. `subagent_wait` joins the thread and returns the result. Intermediate output stays out of the parent's context window, so delegation does not pollute the parent conversation.
Session	Append-only JSONL log with periodic checkpoints. Interactive TUI selector with arrow-key navigation, timestamps, and message counts. Supports rename, tag, delete, and restore. Old single-file format auto-migrated, making the storage layer backward compatible.
Memory	6 typed files under `.agent/memory/`. The prompt asks the agent to proactively remember durable user facts, preferences, project workflows, and explicit corrections. `/memory` performs manual consolidation. On startup, the agent injects only selected fact/preference memories into the system prompt, so stable user identity and preferences are available in a new session without loading the entire memory store.
Skill	Progressive disclosure: startup injects only skill names and descriptions. `load_skill` fetches the full `SKILL.md` on demand. Project skills override user skills with the same name, giving the runtime a namespace and override model similar to configuration layers.
Web Access	`web_fetch` (curl wrapper) and `web_search` (Bing RSS). For JavaScript-heavy sites, the installed `agent-browser` skill can render pages through Chrome. Private and localhost URLs are blocked, keeping web access useful without exposing local services.
Hooks	Event bus with `HOOK_BEFORE_TOOL` and `HOOK_AFTER_TOOL`. Memory observation is registered as a hook, decoupling it from the executor and making future cross-cutting features easier to add.
Dangerous Command Filter	`bash` tool checks 16 patterns (`rm -rf /`, fork bomb, `curl|sh`, etc.) before execution.
Markdown UI[1]	The UI uses MD4C to render markdown in the terminal and keeps spinner/tool status output synchronized through a dedicated render thread.

4 Challenges and Solutions

Concurrent tool output interleaving. When multiple read-only tools complete simultaneously, their `ui_tool_done` calls could interleave terminal output. The solution: a dedicated render thread

[1]Thanks to cJSON, the lightweight C JSON parser used in this project, and MD4C, the markdown parser used for terminal rendering.

owns all stdout via a mutex-protected event queue; the main thread only posts events. `ui_idle()` acts as a barrier before the main thread reclaims output authority.

Context budget explosion. Early versions triggered summary on every turn after the threshold was crossed, causing an LLM call per turn and sometimes hitting model rate limits. Another bug appeared when a recent browser snapshot was too large: summary kept recent messages, so the context could remain above the 32K window forever. Fix: `policy_offload` now scans all tool messages, including recent large outputs, and `ctx_reclaim` stops after the first policy that actually reduces token count.

Offload recovery loops. If the LLM sees an offload placeholder and calls `read_file` to recover it, the recovered content could be offloaded again next turn. Fix: `is_offload_recovery_read` detects when a `read_file` targets an offload path and skips re-offloading that message.

UTF-8 in tool output. Some shell commands produce invalid UTF-8 (e.g. `xxd`, binary files). This corrupted the JSON body sent to the LLM. Fix: `sanitize_utf8_for_json` in `llm_client.c` replaces invalid sequences with `?` before serialization.

Session migration and selector usability. The session format evolved from a single JSON file to a directory-based checkpoint+log format. `migrate_old_sessions` detects legacy files, converts them to the new layout, and removes the originals transparently on startup. The interactive selector also required practical fixes: empty `0 msg` sessions are filtered, session sizes are shown, and Chinese session names are aligned by UTF-8 display width instead of raw byte length.

Search and browser automation loops. A naive Bing HTML parser failed on current result pages, causing the LLM to repeat `web_search`, `web_fetch`, and offload recovery in a loop. The final version uses Bing RSS (`format=rss`) for structured `title/link/description` entries and explicitly tells the agent not to fetch Bing result pages repeatedly. For SPA pages, the installed `agent-browser` skill renders the page with Chrome; on Linux it must run with `--no-sandbox`, which is documented as an operational constraint.

5 Effort Estimation vs. Reality

Phase	Estimated	Actual	Notes
Part 1 Phase A (LLM + bash)	4h	6h	JSON parsing edge cases
Part 1 Phase B (ReAct loop)	2h	2h	Straightforward extension
Part 1 Phase C (REPL + TUI)	3h	4h	Render thread sync tricky
Part 2 Phase A (Registry)	3h	3h	Clean refactor
Part 2 Phase B (Parallel)	2h	3h	TSan debugging
Part 3 Context Management	4h	7h	Recent huge tool outputs, recovery loops
Extensions (all)	8h	18h	Memory, sessions, skills, browser/web, markdown UI
Total	26h	45h	

The largest surprise was the **interaction surface** between extensions. Session persistence needed to coordinate with context offload (which session directory to use), memory needed hooks in the executor, web/browser outputs changed the behavior of context reclamation, and the system prompt had to dynamically assemble memory, skills, web guidance, and context hints. Each extension was individually simple, but their composition required careful ordering and shared-state management. In retrospect, the project was less about writing isolated C functions and more about designing invariants: every tool result must remain recoverable, every persisted session must be replayable, and every background subsystem must not corrupt the main conversation.

6 Course Improvement Suggestions

1. **Provide a proxy-aware test harness.** Students behind HTTP proxies (common in lab environments) will see mysterious 502 errors. The test runner should unset proxy variables or set `NO_PROXY=localhost` automatically.

2. **Add a concurrency stress test.** The current parallel test checks correctness and coarse speedup, but does not exercise thread-safety under contention. A test that fires 8 concurrent `write_file` calls to the same file would catch missing locks.

3. **Clarify the `tools_advertised` test expectation.** The test checks for an exact 4-tool set, but extensions naturally register more tools. A `superset` check (expected $\subseteq$ actual) would allow extensions without penalty.

4. **Provide a reference mock server protocol.** Understanding the mock's scripted response format consumed significant debugging time. A documented example of the request/response deck format would help.

5. **Separate baseline tests from extension tests.** My final system intentionally registers many extra tools, web access, skills, memory, and hooks. A separate extension phase would let students demonstrate extra engineering work without conflicting with tests that assume the minimal baseline tool set.

6. **Introduce a gradual difficulty curve for extensions.** The jump from "basic agent" to "full coding assistant with memory, sessions, skills, and subagents" is large. Scaffolding one extension as a guided tutorial would improve learning outcomes.

7 Conclusion

The final implementation is more than a minimal ReAct loop: it is a small runtime for terminal-based AI assistance. The core OS ideas from the course map naturally onto the design. The tool registry resembles a syscall table, the executor schedules read-only work in parallel while preserving ordered results, the sandbox enforces path containment, session storage uses crash-resistant atomic persistence, and context management behaves like a memory reclamation layer with lossless offload followed by lossy summary.

The extensions were built to support long-running, stateful use rather than one-shot demos. Memory makes stable user facts and preferences available across new sessions, session directories make conversations replayable and isolate offloaded artifacts, skills provide progressive capability loading without inflating the base prompt, and web/browser support lets the agent inspect external resources when local context is insufficient. These features made the system more complex, but they also forced clearer invariants: tool results must remain recoverable, persisted sessions must be replayable, and background subsystems must not corrupt the main conversation.

The behavior-oriented public tests pass, including context offload and summary (`test-ca`, `test-cb`, and C unit tests).[2] Overall, the project demonstrates how OS-level design principles can make an LLM agent more bounded, recoverable, and maintainable in real interactive use.

Due to the five-page report limit, several visual demonstrations and detailed extension notes are placed in a separate appendix PDF. The appendix is not needed to understand the core design, but it documents the additional work that does not fit in the main report: session screenshots, markdown rendering, prompt injection, memory/skill workflows, and other non-required features.

[2] In the aggregate `make test` run, the remaining public mismatch is the baseline `tools_advertised` check. It asserts that exactly the four required tools are advertised, while this extended version deliberately exposes sixteen tools. This is a strict baseline-interface assertion, not a failure of offload, summary, parallel execution, or file operations.